

Reviewer Pack — Gromov 4-Point δ of XOR-Lifted Row Metric Bounds DT Depth

Entry 5ee62d63ba3c · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-17 10:01:57 UTC

Gromov 4-Point δ of XOR-Lifted Row Metric Bounds DT Depth

Entry ID: 5ee62d63ba3c **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-17 10:01:57 UTC

1. Conjecture

Field A (mathematical branch): Coarse geometry / Gromov 4-point δ -hyperbolicity of finite metric spaces (Gromov 1987; Bridson-Haefliger 1999; Coornaert 2014) — a relational metric invariant from geometric group theory with essentially no presence in communication complexity (search ‘Gromov hyperbolicity’ AND ‘communication complexity’ returns no direct hits; <5 papers in adjacent areas) **Field B** (complexity object): Deterministic two-party communication complexity of XOR-gadget-lifted Boolean functions, in the Raz-McKenzie / Göös-Pitassi-Watson / Hatami-Hosseini-Pitassi query-to-communication lifting framework, accessed through the bipartite row metric of the lifted communication matrix $M_{\{f \circ \text{XOR}_2\}}$

Statement:

Let $f: \{0,1\}^n \rightarrow \{0,1\}$ have deterministic decision-tree depth $d = \text{DT}(f)$, let $g = \text{XOR}_2$ (Alice has $x_i \in \{0,1\}$, Bob has $y_i \in \{0,1\}$, $g(x_i, y_i) = x_i \oplus y_i$), and let $M \in \{0,1\}^{\binom{n}{2} \times 2^n}$ be the lifted matrix $M[x,y] = f(x \oplus y)$. View rows of M as points in $(\{0,1\}^{\binom{n}{2}}, d_H)$. For each 4-tuple of distinct rows (a,b,c,d) compute the three sums $S_1 = d_H(a,b) + d_H(c,d)$, $S_2 = d_H(a,c) + d_H(b,d)$, $S_3 = d_H(a,d) + d_H(b,c)$, sort them $S_{\{1\}} \geq S_{\{2\}} \geq S_{\{3\}}$, and set $\delta(M) = \max \text{ over 4-tuples of } (S_{\{1\}} - S_{\{2\}})/2$; let $\bar{\delta}(M) = \delta(M)/\text{diam}(M)$. Conjecture: for every non-constant f , $\bar{\delta}(M_{\{f \circ \text{XOR}_2\}}) \leq 1 - \text{DT}(f)/(n+1)$, i.e. higher DT-depth forces strictly more 4-point-treelike row geometry; a single f with $\bar{\delta}(M) > 1 - \text{DT}(f)/(n+1)$ refutes it.

Rationale (proposer’s reasoning):

Lifting theorems claim DT structure of f survives into the gadget-amplified bipartite matrix; if so, the inherited combinatorial structure should constrain coarse-geometric quantities of the row Hamming cloud, since high-DT functions composed with XOR_2 build hierarchical query trees that the Gromov 4-point inequality is precisely designed to detect. Crucially δ is a 4-point relational invariant — not a single-number truth-table functional — so it sidesteps the Razborov-Rudich largeness obstruction, and uses only Hamming distances (no ring/polynomial structure), so it should not algebrize. If the bound holds, it gives a coarse-geometric proxy for the lifted communication lower bound via embedding-distortion arguments à la Linial-London-Rabinovich.

Taxonomy category: LIFTING (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): abb3e7499f7f8a25

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across all instances ($n \in \{3, 4, 5\}$, 30 random seeds per n stratified by DT depth, plus fixed f : constant, dictator, AND _{n} , OR _{n} , MAJ _{n} , PARITY _{n} , IP _{n} , random AC⁰), compute $\delta(M) \cdot (n+1) / (n+1 - DT(f))$ for each non-constant f . SUPPORTED iff every instance satisfies $\delta(M) \leq 1 - DT(f)/(n+1)$ (ratio ≤ 1); FALSIFIED by a single counterexample.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.90	SAFE	SAFE
NATURAL PROOFS	SAFE	0.78	SAFE	SAFE
ALGEBRIZATION	SAFE	0.82	SAFE	SAFE
KARP_LIPTON	SAFE	0.92	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): -Gromov hyperbolicity AND communication complexity lifting - four-point condition Hamming metric Boolean function decision tree - delta-hyperbolic finite metric XOR lifted matrix query complexity

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import sys
import random
import math
import itertools
from collections import defaultdict

def compute_dt(f, n):
    if n == 0:
        return 0
    for i in range(n):
        fixed_f = defaultdict(int)
        for x in itertools.product([0, 1], repeat=n):
            x_list = list(x)
            x_list[i] = 0
            fixed_f[tuple(x_list)] = f[tuple(x)]
            x_list[i] = 1
            fixed_f[tuple(x_list)] = f[tuple(x)]
```

```

        if all(fixed_f[x] == f[x] for x in itertools.product([0, 1], repeat=n)):
            return compute_dt(fixed_f, n-1)
    return n

def hamming_distance(x, y):
    return sum(1 for a, b in zip(x, y) if a != b)

def run_trial(seed):
    random.seed(seed)
    n = random.choice([3, 4, 5])
    f = defaultdict(int)
    for x in itertools.product([0, 1], repeat=n):
        f[x] = random.randint(0, 1)

    d = compute_dt(f, n)
    if d == 0:
        return {
            "metric_name": "delta_bar_ratio",
            "metric_value": 0.0,
            "instances_tested": 1,
            "conjecture_holds": True,
            "counterexample": ""
        }

    M = []
    for x in itertools.product([0, 1], repeat=n):
        row = []
        for y in itertools.product([0, 1], repeat=n):
            xor = tuple(a ^ b for a, b in zip(x, y))
            row.append(f[xor])
        M.append(row)

    distances = []
    for i in range(len(M)):
        for j in range(i+1, len(M)):
            distances.append(hamming_distance(M[i], M[j]))

    delta = 0
    count = 0
    for a, b, c, d in itertools.combinations(range(len(M)), 4):
        s1 = distances[sum(range(a+1, b+1))] + distances[sum(range(c+1, d+1))]
        s2 = distances[sum(range(a+1, c+1))] + distances[sum(range(b+1, d+1))]
        s3 = distances[sum(range(a+1, d+1))] + distances[sum(range(b+1, c+1))]
        sorted_s = sorted([s1, s2, s3], reverse=True)
        delta = max(delta, (sorted_s[0] - sorted_s[1]) / 2)
        count += 1

    diam = max(distances)
    delta_bar = delta / diam if diam != 0 else 0
    ratio = delta_bar * (n + 1) / (n + 1 - d) if (n + 1 - d) != 0 else float('inf')

    conjecture_holds = ratio <= 1
    counterexample = "" if conjecture_holds else f"delta_bar_ratio={ratio} > 1"

    return {
        "metric_name": "delta_bar_ratio",

```

```

        "metric_value": ratio,
        "instances_tested": count,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97]

    metric_values = []
    conjecture_holds_counts = 0
    total_instances = 0

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        metric_values.append(result["metric_value"])
        conjecture_holds_counts += 1 if result["conjecture_holds"] else 0
        total_instances += result["instances_tested"]

    mean = sum(metric_values) / len(metric_values) if metric_values else 0
    std = math.sqrt(sum((x - mean) ** 2 for x in metric_values) / len(metric_values)) if metric_values else 0
    support_fraction = conjecture_holds_counts / len(seeds) if seeds else 0

    if all(result["conjecture_holds"] for result in [run_trial(seed) for seed in seeds]):
        print(f"RESULT: SUPPORTED mean={mean} std={std} support_fraction={support_fraction}")
    else:
        for seed in seeds:
            result = run_trial(seed)
            if not result["conjecture_holds"]:
                print(f"RESULT: FALSIFIED counterexample={result['counterexample']} first_failing_seed={seed}")
                break

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f5a45145.py", line 97, in <module>
    result = run_trial(seed)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f5a45145.py", line 69, in run_trial
    s3 = distances[sum(range(a+1, d+1))] + distances[sum(range(b+1, c+1))]
              ~~~~~^~~~~~
IndexError: list index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with an `IndexError` before producing any data, so neither the support condition nor a counterexample was established. Per the rules, a crash yields INCONCLUSIVE. | next: Fix the indexing bug in the 4-tuple distance lookup (use a proper pair-to-index mapping like `distances[(i,j)]` or `itertools.combinations` indexing) and rerun the pre-registered sweep.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	309629
2	preregistration	claude_max	opus	0	0	7066
3	novelty	claude_max	opus	0	0	3633
4	novelty	claude_max	opus	0	0	6794
5	test_gen	mistral	codestral-latest	0	0	308281
6	test_gen	mistral	codestral-latest	0	0	317612
7	test_gen	mistral	codestral-latest	0	0	309967
8	test_gen	mistral	codestral-latest	0	0	315911
9	judge	claude_max	opus	0	0	4701

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 1583593 ms total latency. Provider mix: {'claude_max': 5, 'mistral': 4}

(full prompt+response transcripts available in `research/audit/5ee62d63ba3c.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/5ee62d63ba3c.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/5ee62d63ba3c.tar.gz` (if generated)